Solven Website – Complete Frontend Architecture & Component Documentation

A complete technical breakdown of the Solven website frontend architecture, including React components, animations, UI design patterns, libraries used, and implementation details.
This documentation explains how each section of the website works, what technologies are used, and the key concepts developers can learn from the project.

1️⃣ First Understand What the Website Does

The site is a **modern AI SaaS landing page** that explains a product which automates customer service using AI agents that can handle chats, calls, and integrations with POS, CRM, and other systems.

It mainly contains sections like:

Hero
Clients
Features
Product Demo
Stats
Services
About
CTA
Footer

These are typical sections in **startup SaaS websites**.

2️⃣ Technologies Used in the Website

Core Stack

Technology	Purpose
React	Frontend framework
Vite	Fast development server
JavaScript	Logic
CSS	Styling
Framer Motion	Animations
Lucide Icons	Icon library

3️⃣ Libraries You Need to Install

React Project Setup

Create project:

```
npm create vite@latest solven-landing
cd solven-landing
npm install
```

Install Required Libraries

1️⃣ Framer Motion

For animations

npm install framer-motion

Used for:

- scroll animation
- fade in
- scale animation
- hover animation

Example:

```
import { motion } from "framer-motion"

<motion.div
initial={{opacity:0}}
animate={{opacity:1}}
>
Hello
</motion.div>
```

What you learn:

- animation states
- transitions
- motion components

2️⃣ Lucide Icons

For icons.

Install:

npm install lucide-react

Example:

```
import { Rocket } from "lucide-react"

<Rocket size={32} />
```

What you learn:

- SVG icons
- scalable UI

3️⃣ React Intersection Observer

Used for **detecting when element enters screen**

npm install react-intersection-observer

Example:

```
const {ref, inView} = useInView()

if(inView){
 startAnimation()
}
```

What you learn:

- scroll detection
- lazy animation

4️⃣ Folder Structure

```
src
├ components
│   ├ Hero.jsx
│   ├ Clients.jsx
│   ├ Features.jsx
│   ├ ProductDemo.jsx
│   ├ Stats.jsx
│   ├ Services.jsx
│   ├ About.jsx
│   ├ Testimonials.jsx
│   ├ CTA.jsx
│   ├ Footer.jsx
│
├ layout
│   └ Layout.jsx
│
├ hooks
│   └ useScrollToTop.js
│
├ App.jsx
└ main.jsx
```

What you learn:

- component architecture
- reusable UI

5️⃣ Core Concepts that I learn From This Website

1️⃣ Component Based Architecture

React splits UI into small parts.

Example:

Hero
Stats
Services
Footer

Each is independent.

Example:

```
function Hero(){
return <section>Hero Section</section>
}
```

What you learn:

- modular development
- reusable components

6️⃣ Props Concept

Props pass data into components.

Example:

```
<ServiceCard
title="AI Automation"
description="Automate workflows"
/>
```

Inside component:

```
function ServiceCard({title, description}){
return <h3>{title}</h3>
}
```

What you learn:

- data flow
- reusable UI

7️⃣ State Management

React state stores dynamic data.

Example:

```
const [count, setCount] = useState(0)
```

Used in **Stats counter animation**.

What you learn:

- UI updates
- dynamic content

8️⃣ Animation System

Using Framer Motion

Example:

```
<motion.div
initial={{opacity:0, y:50}}
animate={{opacity:1, y:0}}
transition={{duration:0.6}}
>
```

What you learn:

- animation lifecycle
- UI transitions

9️⃣ Scroll Based Animation

Animation triggers when user scrolls.

Example:

```
const {ref, inView} = useInView()

if(inView){
startAnimation()
}
```

Used in:

Stats
Services

What you learn:

- intersection observer
- performance optimization

🔟 Data Driven UI

Instead of writing cards manually.

Example:

```
const services = [
{title:"AI automation"},
{title:"Customer support AI"},
{title:"Analytics"}
]
```

Then render:

```
services.map(service => (
<Card title={service.title}/>
))
```

What you learn:

- dynamic rendering
- map function

1️⃣1️⃣ Responsive Design

Uses CSS grid or flexbox.

Example:

```
grid-template-columns: repeat(auto-fit, minmax(250px,1fr))
```

What you learn:

- responsive UI
- mobile support

1️⃣2️⃣ Orbital Animation (About Section)

This is advanced UI math.

Position items around circle.

Formula:

```
x = center + radius * cos(angle)
y = center + radius * sin(angle)
```

Example:

```
const x = 50 + radius * Math.cos(radians)
const y = 50 + radius * Math.sin(radians)
```

What you learn:

- trigonometry in UI
- interactive design

1️⃣3️⃣ Counter Animation (Stats)

Used to animate numbers.

Example:

0 → 50
0 → 200
0 → 99%

Code concept:

setInterval
increment value
update state

What you learn:

- timers
- animated numbers

1️⃣4️⃣ Testimonials Grid

Typical structure:

Card
├ Rating stars
├ Quote
├ Author

Concepts learned:

- reusable card design
- UI composition

1️⃣5️⃣ CTA Section

Call To Action.

Example:

Start Demo
Contact Sales

Purpose:

Convert visitors to customers.

1️⃣6️⃣ Footer

Contains:

Links
Social media
Contact info

Concept learned:

- website navigation

- SEO structure

1️⃣7️⃣ Smooth Scroll Navigation

Navigation scrolls to sections.

Example:

Home
Features
Services
Contact

Code:

```
document.getElementById("services")
.scrollIntoView({behavior:"smooth"})
```

What you learn:

- DOM manipulation

1️⃣8️⃣ Hooks Concept

React Hooks used:

useState
useEffect
useRef

Example:

```
useEffect(()=>{
fetchData()
},[])
```

What you learn:

- lifecycle control

1️⃣ 9️⃣ Performance Optimization

Used techniques:

lazy animation
component splitting
intersection observer

What you learn:

- performance optimization

2️⃣ 0️⃣ Professional UI Design Concepts

You learn:

SaaS landing page structure
UX hierarchy
visual storytelling
conversion design

💡 What You Learn From Building This Project

Skills gained:

Frontend Skills

- React architecture
- component design
- animation

- responsive design

UI Skills

- SaaS design
- conversion UI
- micro interactions

Advanced Skills

- orbital UI math
- scroll animation
- performance optimization